Solucionario de examen parcial

Problema 1

a. La inserción no desplaza el cursor. Los dos avances ocurren después de insertar 42: $36 \to 42 \to 48$. Al eliminar 48, el cursor pasa a su sucesor, 60.

Operación	Secuencia lógica	Cursor
Inicial	12,24,36,48,60,72,84	36 (2)
`insertAfterCurrent(42)`	12,24,36,42,48,60,72,84	36 (2)
Avanzar dos posiciones	Sin cambio	48 (4)
`removeCurrent()`	12,24,36,42,60,72,84	60 (4)
`insertAfterCurrent(66)`	12,24,36,42,60,66,72,84	60 (4)
`get(5)`	Sin cambio, devuelve **66**	60 (4)

Arreglo final: $a[0..7] = [12, 24, 36, 42, 60, 66, 72, 84]$, $n = 8$, capacidad 8, `currentIndex=4`. En DLList, `current` apunta al *nodo* 60, no a un índice. No ocurre `resize`.

b. Movimientos en el arreglo: insertar 42 desplaza $48, 60, 72, 84$ (4); eliminar 48 desplaza $60, 72, 84$ (3); insertar 66 desplaza $72, 84$ (2): **9 desplazamientos**. En DLList, cada inserción asigna cuatro enlaces ($u.prev$, $u.next$ y los dos de sus vecinos); eliminar asigna dos: $4 + 2 + 4 = 10$ **escrituras a `prev`/`next`**. No se cuentan creación de nodos, tamaño ni cambio de cursor. Localizar las modificaciones desde `current` es $O(1)$; mover el cursor requiere dos saltos `next`. Para `get(5)`, desde el último nodo 84 se recorren dos enlaces `prev`: $84 \to 72 \to 66$ (más el acceso inicial a `dummy.prev`, si se cuenta).

c. Se guardan ambos vecinos *antes* de desconectar el nodo; los casos de único nodo y último nodo están cubiertos.

```
void removeCurrent() {
    Node u = current;
    Node p = u.prev, s = u.next;
    p.next = s;
    s.prev = p;
    n--;
    if (n == 0) current = dummy;
    else current = (s != dummy) ? s : p;
}
```

d. Arreglo: $0 \le n \le a.length$ y los n elementos, en orden, ocupan $a[0..n-1]$; un desplazamiento defectuoso podría perder un valor. DLList: (i) para cada nodo activo, $u.next.prev = u$ y $u.prev.next = u$, (ii) recorrer desde `dummy.next` regresa a `dummy` tras exactamente n nodos; ambos detectan enlaces rotos o nodos inaccesibles. Para 85 % de modificaciones locales conviene **DLList**, por reconexiones constantes; para 85 % de `get(i)` arbitrarios conviene **arreglo**, con acceso $O(1)$ frente a $O(n)$ de la lista.

Problema 2

a. En la cola circular, el elemento lógico k está en $a[(j+k) \bmod m]$ y el siguiente hueco de inserción en $(j+n) \bmod m$.

Evento	Posiciones físicas escritas/liberadas	j	n	FIFO después del evento
Inicial	5:T1, 6:T2, 7:T3, 0:T4, 1:T5	5	5	T1,T2,T3,T4,T5
1. T1 termina	$a[5] \leftarrow null$; apilar T1	6	4	T2,T3,T4,T5
2. T2 rota	$a[6] \leftarrow null$; $a[2] \leftarrow T2$	7	4	T3,T4,T5,T2
3. T6	$a[3] \leftarrow T6$	7	5	T3,T4,T5,T2,T6
4. T7	$a[4] \leftarrow T7$	7	6	T3,T4,T5,T2,T6,T7
5. T8	$a[5] \leftarrow T8$	7	7	T3,T4,T5,T2,T6,T7,T8
6. T9	$a[6] \leftarrow T9$	7	8	T3,T4,T5,T2,T6,T7,T8,T9
7. T10	Copia $a \to b[0..7]$; $b[8] \leftarrow T10$	0	9	T3,T4,T5,T2,T6,T7,T8,T9,T10

En el evento 2, el retiro deja provisionalmente $j = 7, n = 3$, y solo después se inserta T2. Los tres estados requeridos son:

- Antes del crecimiento: $m = 8, j = 7, n = 8$, $a[0..7] = [T4, T5, T2, T6, T7, T8, T9, T3]$.
- Después del crecimiento, antes de T10: $m = 16, j = 0, n = 8$, $b[0..7] = [T3, T4, T5, T2, T6, T7, T8, T9]$ y $b[8..15] = null$.
- Después de T10: $m = 16, j = 0, n = 9$, $a[0..8] = [T3, T4, T5, T2, T6, T7, T8, T9, T10]$; restantes `null`. Pila: `head -> T1 -> null`, $n = 1$.

b. Se copia el orden *lógico*, no la disposición física anterior:

```
private void resize() {
    Task[] b = new Task[Math.max(1, 2*n)];
    for (int k = 0; k < n; k++)
        b[k] = a[(j+k) % a.length];
    a = b;
    j = 0;
}
```

Aquí `Task` representa el tipo concreto almacenado por la cola. Al fijar $j' = 0$, el elemento lógico k queda en $b[k]$, preservando FIFO.

c. Antes del crecimiento $j = 7$; el frente correcto es $a[7] = T3$. La copia incorrecta $b[0] = a[0] = T4$ haría que `peek()` y el siguiente `remove()` observaran **T4**, no T3. Por ello el defecto cambia la semántica FIFO, no solamente su representación.

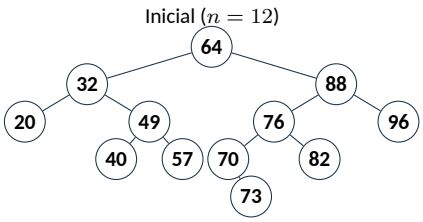
d. En la pila enlazada:

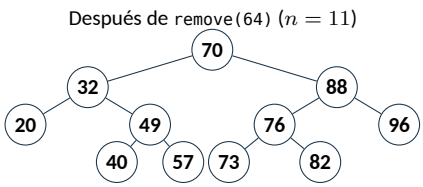
```
void push(T x) {
    Node u = new Node();
    u.x = x;
    u.next = head;
    head = u;
    n++;
}
```

La nueva cabeza enlaza con los nodos anteriores: se conservan n nodos alcanzables desde `head`; el costo es $O(1)$. Sin crecimiento, `ArrayQueue.add` es $O(1)$. Una llamada con `resize` cuesta $O(n)$, pero el crecimiento geométrico permite $O(1)$ *amortizado*.

Problema 3

a. El BST simple (sin rotaciones) queda definido por las siguientes relaciones de hijos; se muestran también las estructuras inicial y posterior al retiro de 64:





$70.parent = 76$, $73.parent = 70$, $76.parent = 88$. Inorder inicial: $20, 32, 40, 49, 57, 64, 70, 73, 76, 82, 88, 96$.

b. En `remove(64)`, la raíz tiene dos hijos. El sucesor inorder se localiza por $64 \to 88 \to 76 \to 70$ (mínimo del subárbol derecho). El sucesor físico 70 no tiene hijo izquierdo pero **sí tiene hijo derecho 73**. Se copia 70 en `root.x`; luego `splice` cambia `76.left` del antiguo nodo 70 al nodo 73 y `73.parent` de 70 a 76. No cambian la referencia `root` ni `88.left`. Así 73 conserva su acceso desde la raíz y todos los demás subárboles permanecen conectados; n pasa de 12 a 11.

c. `remove(20)` encuentra una hoja: devuelve `true`, deja `32.left=null` y $n = 10$. `remove(21)` devuelve `false` y mantiene $n = 10$: `findLast(21)` termina en 32, pero $32 \neq 21$. Inorder final: $32, 40, 49, 57, 70, 73, 76, 82, 88, 96$.

d. La precondición de `splice` es que `u` posee cero o un hijo; el método no administra el contador:

```
private void splice(Node u) {
    Node child = u.left != null ? u.left : u.right;
    Node p = u.parent;
    if (p == null) root = child;
    else if (p.left == u) p.left = child;
    else p.right = child;
    if (child != null) child.parent = p;
}
```

Si es hoja, el enlace se hace nulo; si tiene un hijo, se conecta al padre original; si es la raíz, se actualiza `root` y el nuevo padre queda nulo. `splice` es $O(1)$ y `remove` disminuye n una sola vez incluso cuando hay sucesor.

e. `findLast` sigue un camino de a lo más $h + 1$ nodos: $O(h + 1)$. `contains` lo reutiliza y compara la clave; `add` añade $O(1)$ enlaces después de buscar; `remove` localiza el nodo y, si tiene dos hijos, el sucesor, y ejecuta `splice` $O(1)$. Todas: $O(h + 1)$, usualmente $O(h)$ para $h \geq 1$. Si el BST degenera, $h = n - 1$ y resultan $O(n)$; $O(\log n)$ solo vale cuando la altura es logarítmica, algo que el BST simple no garantiza.

Problema 4

a. Después de $30, 100, 10, 15, 120, 70, 95, 60$ no hay rotaciones. El AVL tiene enlaces $30 \to (10, 100)$, $10.right = 15$, $100 \to (70, 120)$ y $70 \to (60, 95)$. Metadatos $(height, balanceFactor)$: $15, 60, 95, 120 : (1, 0)$; $10 : (2, -1)$; $70 : (2, 0)$; $100 : (3, +1)$; $30 : (4, -1)$. Todos los factores pertenecen a $[-1, 1]$ y los prefijos de inserción mantienen el balance.

b. El recorrido de `add(90)` es $30 \to 100 \to 70 \to 95 \to$ `left null`. Al volver: $bf(95) = +1$, $bf(70) = -1$ y $bf(100) = +2$; el primer ancestro desbalanceado es **100**, caso **LR**. Se ejecuta `100.left=rotateLeft(70)` y después `rotateRight(100)`. En la primera rotación se guarda el subárbol intermedio `95.left = 90` y se reconecta como `70.right=90`; la nueva raíz local es 95, que se asigna a `30.right`. Estructura final: $30 \to (10, 95)$, $10.right = 15$, $95 \to (70, 100)$, $70 \to (60, 90)$, $100.right = 120$. Alturas finales: hojas $= 1$, $70 = 2$, $100 = 2$, $95 = 3$, raíz $30 = 4$; factores: $70 = 0$, $100 = -1$, $95 = 0$, $30 = -1$.

c. Se utilizan las rotaciones suministradas; el valor retornado sustituye siempre la raíz local correspondiente:

```
private Node rebalance(Node u) {
    int f = balanceFactor(u);
    if (f > 1) {
        if (balanceFactor(u.left) < 0)
            u.left = rotateLeft(u.left); // LR
        return rotateRight(u);         // LL o LR
    }
    if (f < -1) {
        if (balanceFactor(u.right) > 0)
            u.right = rotateRight(u.right); // RL
        return rotateLeft(u);              // RR o RL
    }
    return u;
}
```

d. Un inorder creciente certifica el orden BST, no el balance. Deben verificarse, para cada nodo, dos invariantes adicionales:
$u.height = 1 + \text{máx}(height(u.left), height(u.right))$ y $|balanceFactor(u)| \leq 1$. Una rotación modifica $O(1)$ enlaces y alturas: $O(1)$. La inserción AVL recorre un camino de longitud $O(\log n)$, con trabajo constante en cada retorno: $O(\log n)$.

Problema 5

a. En `heapify`, $n = 10$, último nodo interno $\lfloor n/2 \rfloor - 1 = 4$; se procesa de 4 a 0:

i	Intercambios	Arreglo al terminar `trickleDown(i)`
4	Ninguno	17,9,24,5,12,20,28,3,7,15
3	(3,7)	17,9,24,3,12,20,28,5,7,15
2	(2,5)	17,9,20,3,12,24,28,5,7,15
1	(1,3),(3,7)	17,3,20,5,12,24,28,9,7,15
0	(0,1),(1,3),(3,8)	**3,5,20,7,12,24,28,9,17,15**

Árbol completo representado por el arreglo: la raíz 3 tiene hijos 5 y 20; el 5 tiene 7 y 12; el 20 tiene 24 y 28; el 7 tiene 9 y 17; el 12 tiene hijo izquierdo 15. Se verifica $3 \leq 5, 20$; $5 \leq 7, 12$; $20 \leq 24, 28$; $7 \leq 9, 17$; $12 \leq 15$. Todos los elementos ocupan el segmento $a[0..9]$ sin huecos.

b. `add(4)` provoca crecimiento $10 \to 20$. `bubbleUp`: $(10, 4)$ y $(4, 1)$.

$$a[0..10] = [3, 4, 20, 7, 5, 24, 28, 9, 17, 15, 12], \quad n = 11.$$

Primer `remove`: devuelve **3**, sube 12 (índice 10), intercambia $(0, 1), (1, 4)$.

$$a[0..9] = [4, 5, 20, 7, 12, 24, 28, 9, 17, 15], \quad n = 10.$$

Segundo `remove`: devuelve **4**, sube 15 (índice 9), intercambia $(0, 1), (1, 3), (3, 7)$.

$$a[0..8] = [5, 7, 20, 9, 12, 24, 28, 15, 17], \quad n = 9.$$

La capacidad permanece en **20**: no se activa reducción. Se elimina la última posición para conservar la forma de árbol completo; quitar una posición interior dejaría un hueco.

c. El algoritmo comprueba límites, compara con los hijos válidos y baja hasta cumplir la propiedad mínima:

```
private void trickleDown(int i) {
    while (true) {
        int best = i;
        int l = 2*i + 1, r = 2*i + 2;
        if (l < n && a[l] < a[best]) best = l;
        if (r < n && a[r] < a[best]) best = r;
        if (best == i) return;
        swap(i, best);
        i = best;
    }
}
```

d. A altura t desde las hojas hay $O(n/2^{t+1})$ nodos; cada uno desciende como máximo t niveles. Por tanto,

$$T(n) \leq O\left(n \sum_{t \geq 0} \frac{t}{2^{t+1}}\right) = O(n), \qquad \sum_{t \geq 0} \frac{t}{2^{t+1}} = 1.$$

Por inserciones sucesivas, cada `bubbleUp` tiene cota $O(\log n)$ y n inserciones tienen cota $O(n \log n)$. `peek` cuesta $O(1)$; `add` y `remove`, $O(\log n)$ amortizado considerando el crecimiento o reducción geométricos, aunque una llamada con `resize` puede costar $O(n)$. `ArrayQueue` retira por antigüedad (FIFO), no por menor severidad, como exige la cola de prioridad. **También se considera válida una explicación mediante un gráfico** que represente la cantidad de nodos a cada altura y el trabajo acumulado, siempre que permita justificar la cota $O(n)$.